

```
In [40]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

from sklearn.svm import SVC # Support vector machines
```

```
In [41]: # Load training and test data
train_data = pd.read_csv('data/train.csv')
X_test = pd.read_csv('data/test.csv')

# Separate features and target in training data
X_train = train_data.drop('Quality', axis=1)
y_train = train_data['Quality']

print("Training data shape:", train_data.shape)
print("Test data shape:", X_test.shape)
```

Training data shape: (2800, 10)

Test data shape: (1200, 9)

```
In [42]: # Check class distribution in the target variable
plt.figure(figsize=(4, 2))
sns.countplot(x=y_train)
plt.title('Distribution of Banana Quality Classes')
plt.xlabel('Quality (0 = Bad, 1 = Good)')
plt.ylabel('Count')
plt.show()

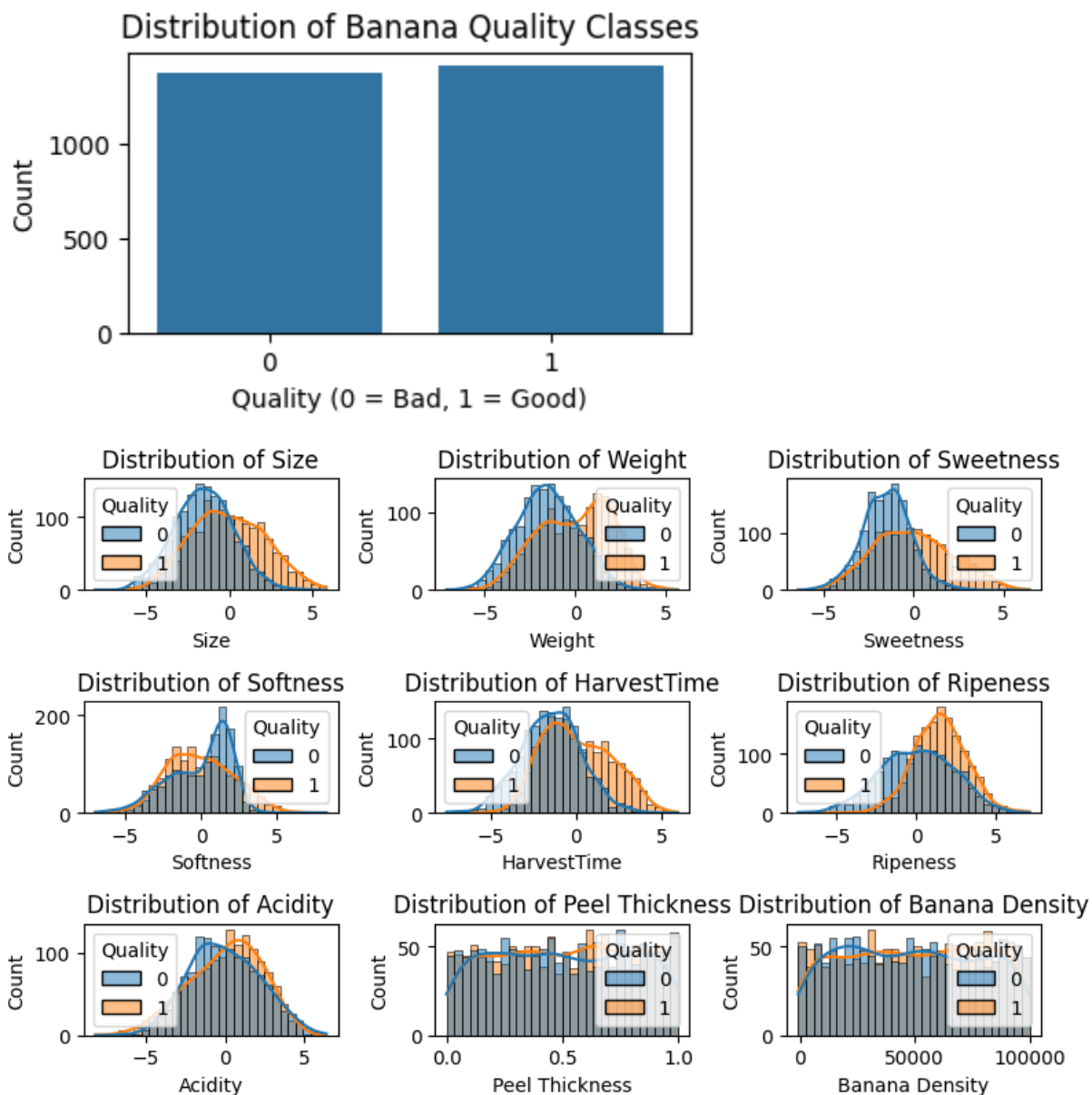
# Visualize feature distributions
plt.figure(figsize=(8, 5))
for i, column in enumerate(X_train.columns):
    plt.subplot(3, 3, i+1)
    sns.histplot(data=X_train, x=column, hue=y_train, kde=True, bins=30)
    plt.title(f'Distribution of {column}')
plt.tight_layout()
plt.show()

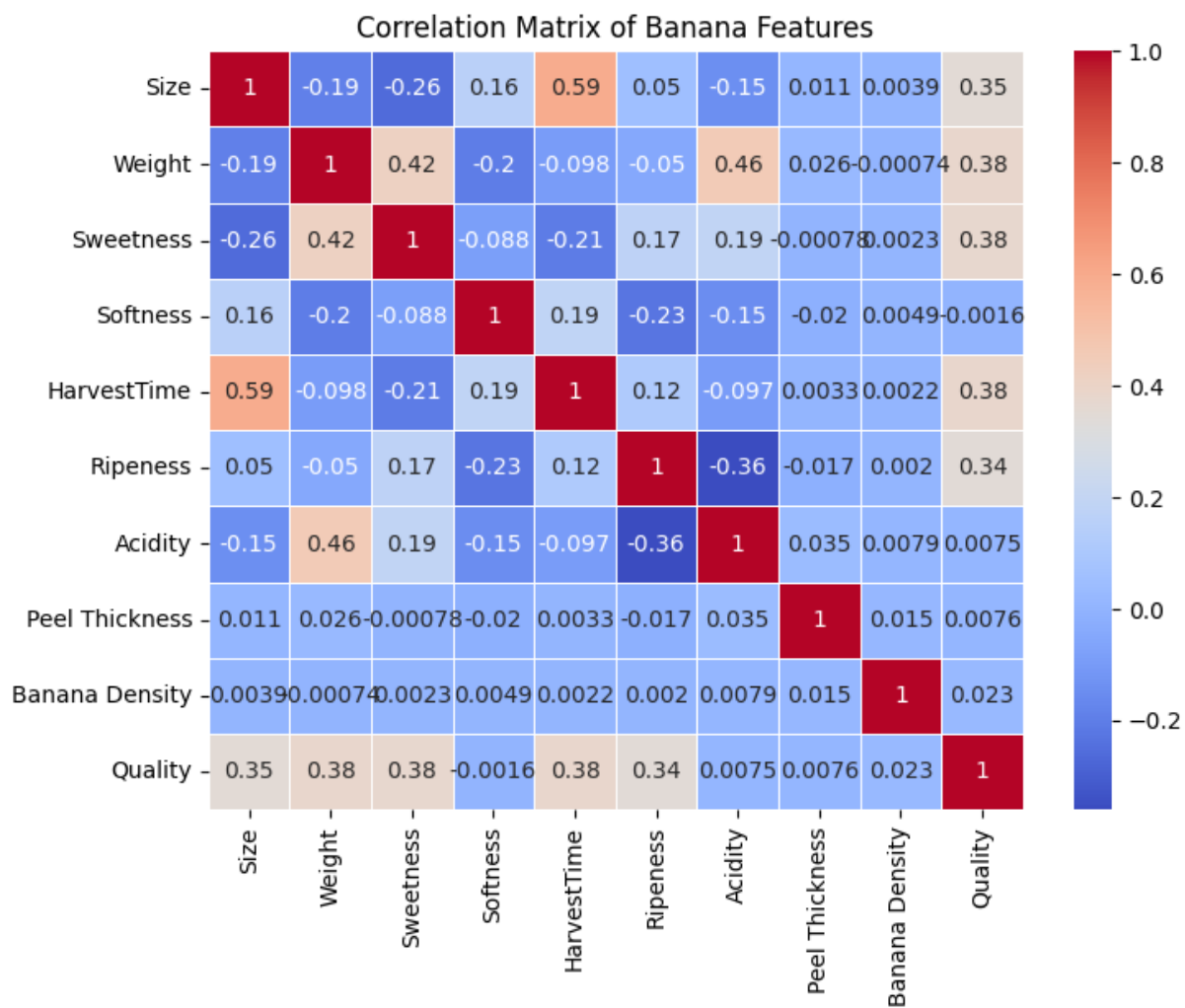
# Create correlation matrix
plt.figure(figsize=(8, 6))
correlation_matrix = train_data.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix of Banana Features')
plt.show()

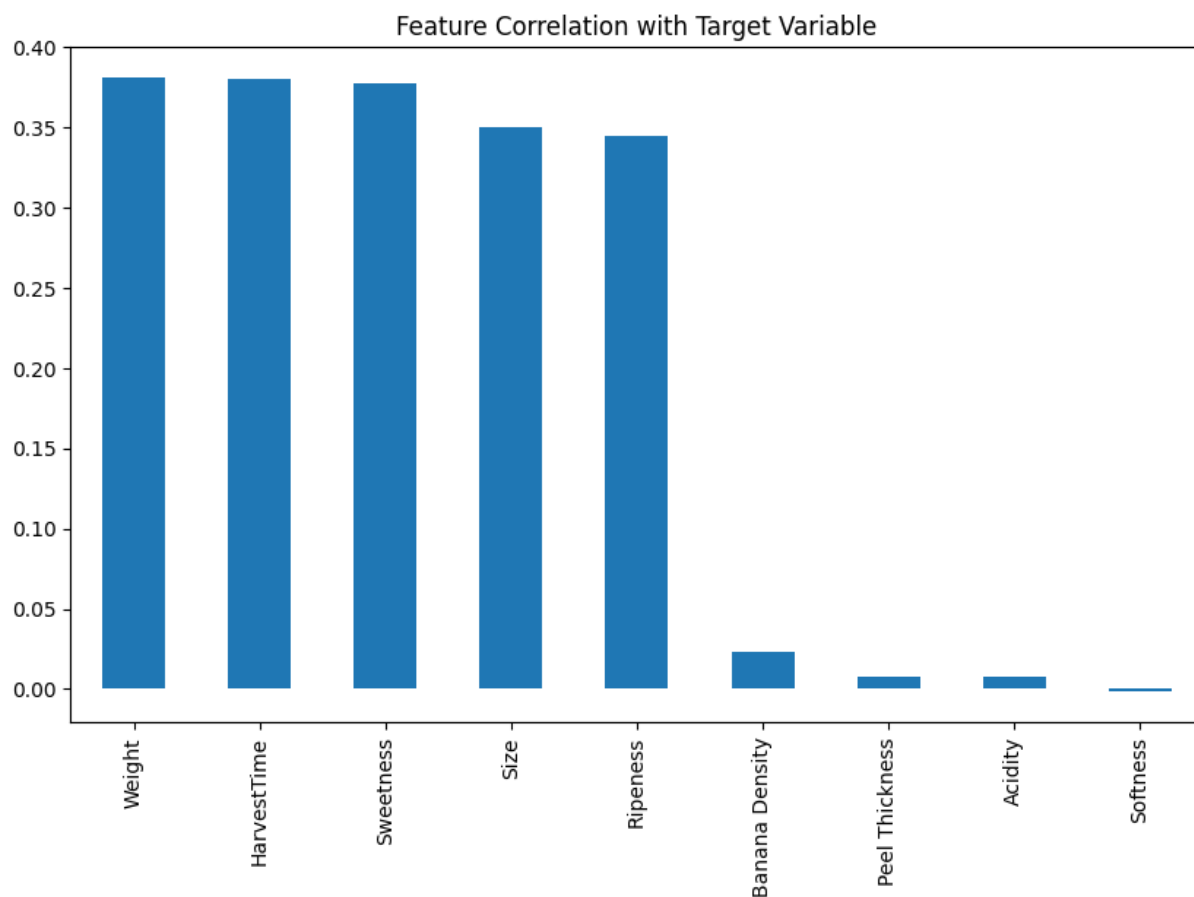
# Correlation with target variable
correlations = X_train.corrwith(y_train).sort_values(ascending=False)
```

```
plt.figure(figsize=(10, 6))
correlations.plot(kind='bar')
plt.title('Feature Correlation with Target Variable')
plt.show()

print("Feature correlations with Quality:")
print(correlations)
```







Feature correlations with Quality:

```

Weight          0.381014
HarvestTime     0.380325
Sweetness       0.377916
Size            0.350030
Ripeness        0.344412
Banana Density  0.023014
Peel Thickness  0.007586
Acidity         0.007544
Softness        -0.001552
dtype: float64

```

The data is nicely balanced in 0 classifications and 1 classifications. 5 Features seem to be the most influential with the 4 others having a much less correlation to quality. However with feature selection experimentation we tried removing some and the model performed worse. So we chose not to delete any features later on.

```

In [43]: # Function to remove outliers
def remove_outliers(X, y, threshold=3): # threshold = 3 is default, as in 1
    z_scores = np.abs((X - X.mean()) / X.std())
    mask = (z_scores < threshold).all(axis=1)
    return X[mask], y[mask]

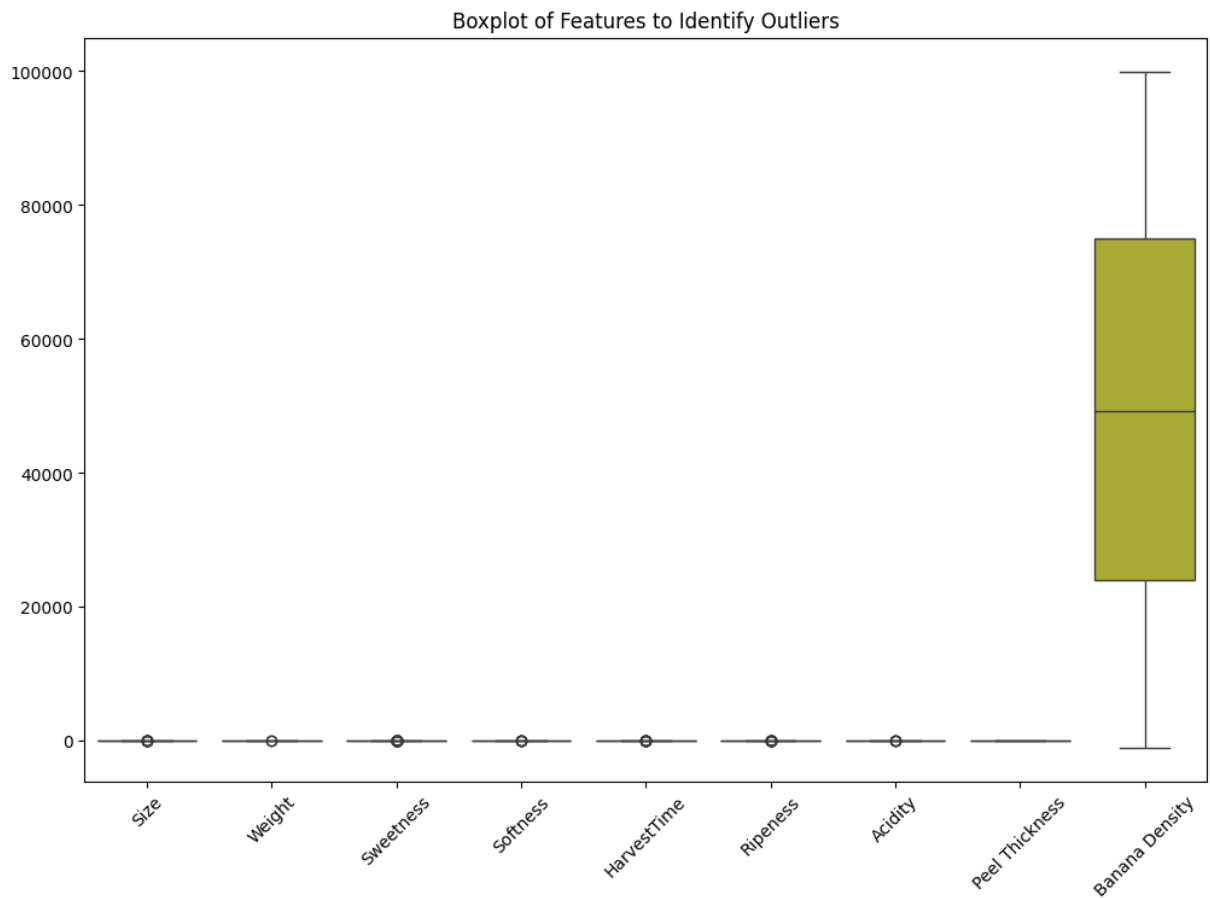
# Check for outliers
plt.figure(figsize=(12, 8))
sns.boxplot(data=X_train)
plt.title('Boxplot of Features to Identify Outliers')
plt.xticks(rotation=45)

```

```
plt.show()

# Apply outlier removal
X_train_clean, y_train_clean = remove_outliers(X_train, y_train, threshold=3)
print(f"Removed {X_train.shape[0] - X_train_clean.shape[0]} outliers")
print(f>Data shape after outlier removal: {X_train_clean.shape}")

# %%
# Check for missing values
print("\nMissing values in training data:")
print(train_data.isnull().sum())
print("\nMissing values in test data:")
print(X_test.isnull().sum())
```



```
Removed 31 outliers  
Data shape after outlier removal: (2769, 9)
```

```
Missing values in training data:
```

```
Size          0  
Weight        0  
Sweetness     0  
Softness      0  
HarvestTime   0  
Ripeness      0  
Acidity       0  
Peel Thickness 0  
Banana Density 0  
Quality       0  
dtype: int64
```

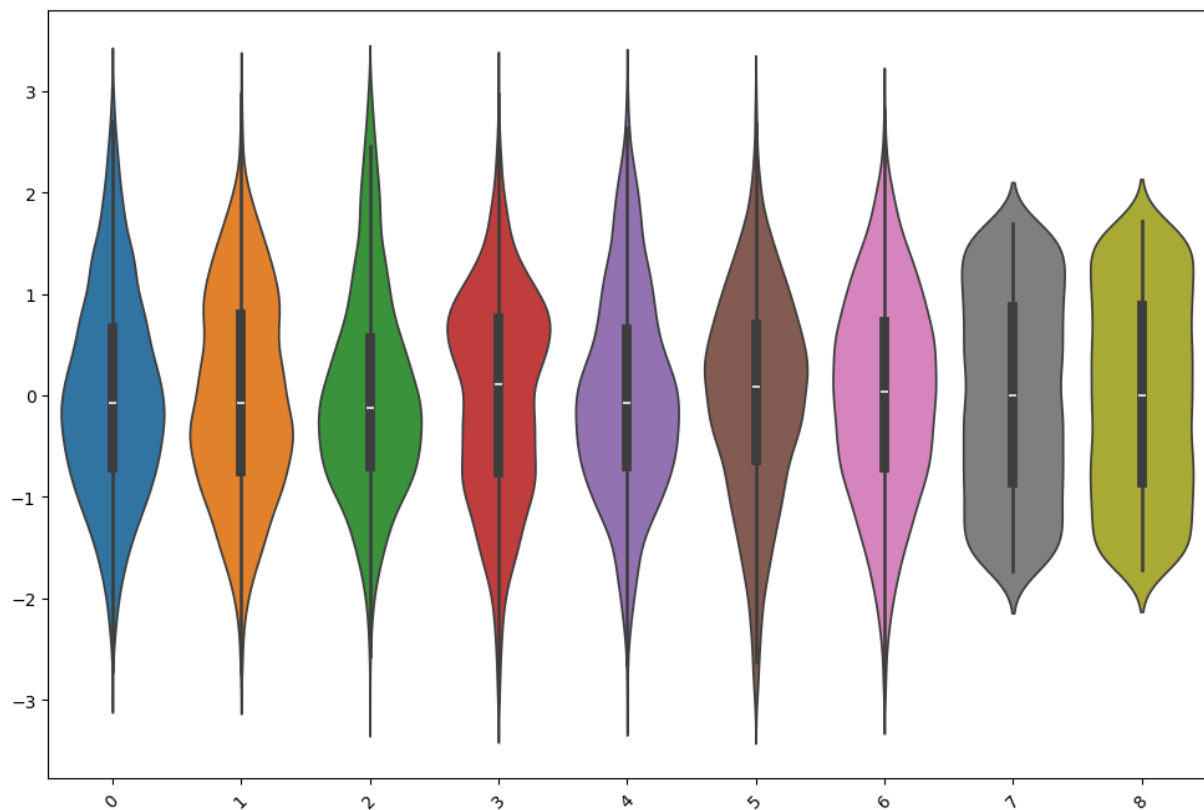
```
Missing values in test data:
```

```
Size          0  
Weight        0  
Sweetness     0  
Softness      0  
HarvestTime   0  
Ripeness      0  
Acidity       0  
Peel Thickness 0  
Banana Density 0  
dtype: int64
```

Picture really only shows that most values are small, and that banana density have a high value in comparison (so we need to scale), hard to spot outliers from this. But the function found some, not many and it didn't seem to do too much. Also showed no missing data. So data looks good for training.

```
In [44]: # Standardize features  
scaler = StandardScaler()  
scaler.fit(X_train_clean)  
X_train_scaled = scaler.transform(X_train_clean)  
X_test_scaled = scaler.transform(X_test)
```

```
In [45]: # Data after scaling and outlier removal  
plt.figure(figsize=(12, 8))  
sns.violinplot(data=X_train_scaled)  
plt.xticks(rotation=45)  
plt.show()
```



```
In [46]: # Function to evaluate a model with multiple train-test splits, get mean acc
def evaluate_model(model, X, y, n_splits=10):
    accuracy_scores = []

    for i in range(n_splits):
        X_t, X_v, y_t, y_v = train_test_split(X, y, test_size=0.2, random_st
        model.fit(X_t, y_t)
        y_pred = model.predict(X_v)
        accuracy_scores.append(accuracy_score(y_v, y_pred))

    return np.mean(accuracy_scores)

best_models = {}

# Train and evaluate a support vector machine
# Trains with different C and gamma values then evaluates to find the values
C_values = [0.01, 0.1, 1, 10, 100, 1000]
gamma_values = [0.001, 0.01, 0.1, 0.5, 1, 2]
best_acc = 0
best_C = None
best_gamma = None

results = []

for C in C_values:
    for g in gamma_values:
        model = SVC(C=C, gamma=g, random_state=42) # default kernel = rbf
        accuracy = evaluate_model(model, X_train_scaled, y_train_clean)
        print(f"C={C}, gamma={g}, Accuracy={accuracy:.4f}")
```

```

        results.append({'C': C, 'gamma': g, 'accuracy': accuracy})

    if accuracy > best_acc:
        best_acc = accuracy
        best_C = C
        best_gamma = g

print(f"Best SVM: C={best_C}, gamma={best_gamma}, Accuracy={best_acc:.4f}")

```

```

C=0.01, gamma=0.001, Accuracy=0.5054
C=0.01, gamma=0.01, Accuracy=0.6792
C=0.01, gamma=0.1, Accuracy=0.9247
C=0.01, gamma=0.5, Accuracy=0.5054
C=0.01, gamma=1, Accuracy=0.5054
C=0.01, gamma=2, Accuracy=0.5054
C=0.1, gamma=0.001, Accuracy=0.7964
C=0.1, gamma=0.01, Accuracy=0.8897
C=0.1, gamma=0.1, Accuracy=0.9605
C=0.1, gamma=0.5, Accuracy=0.9706
C=0.1, gamma=1, Accuracy=0.8554
C=0.1, gamma=2, Accuracy=0.5054
C=1, gamma=0.001, Accuracy=0.8736
C=1, gamma=0.01, Accuracy=0.9130
C=1, gamma=0.1, Accuracy=0.9724
C=1, gamma=0.5, Accuracy=0.9782
C=1, gamma=1, Accuracy=0.9736
C=1, gamma=2, Accuracy=0.9639
C=10, gamma=0.001, Accuracy=0.8830
C=10, gamma=0.01, Accuracy=0.9514
C=10, gamma=0.1, Accuracy=0.9785
C=10, gamma=0.5, Accuracy=0.9733
C=10, gamma=1, Accuracy=0.9736
C=10, gamma=2, Accuracy=0.9652
C=100, gamma=0.001, Accuracy=0.9031
C=100, gamma=0.01, Accuracy=0.9646
C=100, gamma=0.1, Accuracy=0.9682
C=100, gamma=0.5, Accuracy=0.9729
C=100, gamma=1, Accuracy=0.9736
C=100, gamma=2, Accuracy=0.9652
C=1000, gamma=0.001, Accuracy=0.9496
C=1000, gamma=0.01, Accuracy=0.9720
C=1000, gamma=0.1, Accuracy=0.9545
C=1000, gamma=0.5, Accuracy=0.9729
C=1000, gamma=1, Accuracy=0.9736
C=1000, gamma=2, Accuracy=0.9652
Best SVM: C=10, gamma=0.1, Accuracy=0.9785

```

```

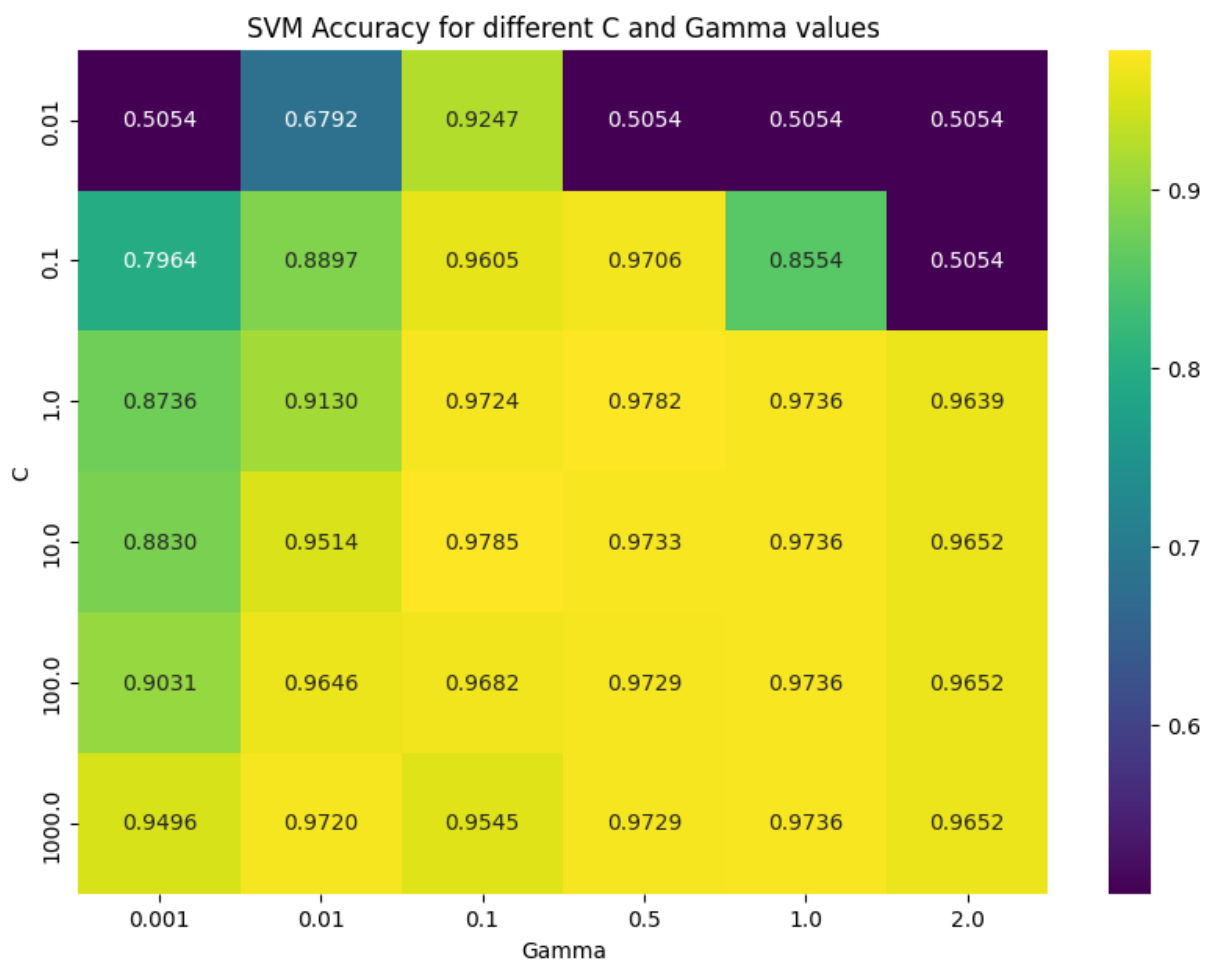
In [47]: # Convert results to a DataFrame
results_df = pd.DataFrame(results)

accuracy_pivot = results_df.pivot(index='C', columns='gamma', values='accuracy')
plt.figure(figsize=(10, 7))
sns.heatmap(accuracy_pivot, annot=True, fmt=".4f", cmap="viridis")
plt.title('SVM Accuracy for different C and Gamma values')
plt.xlabel('Gamma')

```



```
plt.ylabel('C')
plt.show()
```



```
In [48]: best_model = SVC(C=best_C, gamma=best_gamma, random_state=42)
```

```
In [49]: best_model.fit(X_train_scaled, y_train_clean)
y_pred = best_model.predict(X_test_scaled)

submission = pd.DataFrame({'ID': X_test.index, 'Quality': y_pred})
submission.to_csv('submission.csv', index=False)

print("Distribution of target variable:")
print(y_train.value_counts())
```

Distribution of target variable:

Quality

1 1418

0 1382

Name: count, dtype: int64

```
In [ ]:
```